

Performance Evaluation of Tensorflow Parallel Strategies for Deep Learning Training on HPC Clusters

Rayan Raddatz de Matos¹, Kenichi Brumato¹, Marcelo Cardoso Oliveira Gulart¹

¹ Institute of Informatics, Federal University of Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

{rayan.raddatz, kbrumati, mcogulart}@inf.ufrgs.br

Abstract. *The growing complexity of Deep Learning models makes training on single devices difficult, requiring distributed strategies like Data Parallelism. However, this approach creates critical synchronization challenges between processing units. This work evaluates TensorFlow parallel strategies on an HPC cluster to analyze how application factors contribute to time reduction. The study compares the standard synchronous method against a proposed Custom Training Loop designed to reduce bottleneck by synchronizing gradients less frequently. Experiments using ResNet50 and EfficientNetB0 models show that the Custom Training Loop significantly lowers communication overhead compared to the standard approach. Additionally, results indicate that increasing the batch size consistently improves performance. The authors conclude that correct configuration allows for performance gains in distributed training, even on networks with lower throughput.*

Resumo. *Coloque o resumo aqui.*

1. Introduction

In the last decade, Deep Learning has established itself as the state-of-the-art for solving complex problems, such as computer vision and natural language processing. Given the progress in algorithmic precision, there has been an exponential growth in the complexity of neural network architectures and the volume of datasets. This scenario imposes a massive computational load, making the training of robust models on a single processing unit an difficult task in terms of time and resources.

To mitigate these challenges, distributed training has become a standard practice, with an emphasis on the Data Parallelism strategy. In this approach, a complete replica of the model is instantiated on each processing unit, and the global dataset is partitioned into smaller, locally-focused batches. Although effective, this technique introduces critical synchronization challenges, as locally calculated gradients must be aggregated and communicated among all devices at every training step, typically through All-Reduce operations.

Given this fact, in this work we decided to analyze data parallelism for AI training using TensorFlow in a HPC cluster to investigate how the factors related to the application contribute to time reduction in a parallel scenario.

Software and Data Availability. We endeavor to make our analysis reproducible for a better science. We made available a companion material hosted in a public GitHub repository at <https://github.com/kenichi220/comp-sys-perf-analysis-final-work/tree/main>. Our

companion contains the source code of this article and the software necessary to handle the created datasets. We also include instructions to run the experiment and figures.

2. Background and Experimental Context

2.1. Background

TensorFlow is a machine learning system that operates at a large scale and in heterogeneous [Abadi et al. 2016]. It uses dataflow graphs to represent the computation, shared state, and the operations that alter this state.

This architecture maps the nodes of a dataflow graph onto many machines within a cluster and, within a machine, onto multiple computational devices (multicore CPUs, GPUs, and TPUs). This offers flexibility to the developer to experiment with new optimizations and training algorithms, supporting a variety of applications focused on deep neural networks. When it comes to distributed training, TensorFlow offers different strategies with the `tf.distribute.Strategy` API. The TensorFlow documentation advertises that *"For synchronous training on many GPUs on multiple workers, use the `tf.distribute.MultiWorkerMirroredStrategy` with the Keras `Model.fit` or a custom training loop."* [Documentation 2023]. Meaning for a distributed training across different nodes in a cluster there are two options: Using the `MultiWorkerMirroredStrategy` with the `model.fit` or creating a Custom Training Loop by yourself.

2.2. Experimental Context

In High-Performance Computing (HPC) environments, a highly recommended practice to access to shared computational resources is to mediate the access by workload management systems. In our context Slurm (Simple Linux Utility for Resource Management) is utilized. Slurm is an open-source, fault-tolerant, and highly scalable job scheduler, responsible for three main functions: Allocating exclusive or non-exclusive access to resources/nodes for users, providing a framework to initiate, execute, and monitor the work on the set of allocated nodes and managing a queue of pending jobs [Yoo et al. 2003].

Interaction with Slurm is typically performed through shell scripts written in the BASH language. These scripts contain preprocessing directives that specify resource requirements, such as the number of nodes, tasks per node, and execution time limit, enabling the automation and reproducibility of computational experiments.

3. Methods and Materials

3.1. Software Management and Reproducibility

Reproducibility of computational experiments is a critical challenge in computer science. To mitigate software dependency issues, the use of the GNU Guix package manager was explored. Guix implements the functional package management paradigm where each package is installed in a unique and immutable directory in the store, derived from a cryptographic hash of its dependencies [Courtès 2013].

However, the efficient training of deep neural networks using TensorFlow requires the use of proprietary NVIDIA hardware acceleration libraries, which are not officially distributed in standard Guix channels due to its strict Free Software policy.

Given this restriction, the strategy adopted in this work is the use of Python Virtual Environment for environment management. Although a functional version of Guix exists on GitHub without the proprietary libraries.

For managing the specific Deep Learning libraries, the python package manager `pip` with version 25.3 was utilized. The exact version of the used library was frozen and documented using the `pip freeze` command, generating a requirements file (`reqs.txt`) that enables the reconstruction of the execution environment used in the experiments.

3.2. Hardware and Software Configuration

The experiments were run using TensorFlow 2.15 as newer versions showed errors when trying to train using more than one node with the `.fit` and `MultiWorkerMirroredStrategy` and python 3.11.2. The NVIDIA drivers version was 550.54.15 and CUDA 12.4. We used up to three computational nodes connected with a network of 1G, each node contains an Intel(R) Core(TM) i7-14700KF processor at 3.40 GHz with 28 threads and 20 cores, 96 GB DDR5 RAM and NVIDIA GeForce RTX 4070 with 8GB memory accelerator.

3.3. Custom Training Loop

The expected execution from training with the `MultiWorkerMirroredStrategy` would show a slower training time as we increased the number of nodes in the execution. However, we faced a problem due to the connection while training that resulted in a higher training time as we increased the number of nodes for big models with a lot of parameters such as ResNet50. This problem happened because a synchronization is done for every batch computed by the `model.fit()` training method, and as we had only an 1G network, this synchronization resulted in a bottleneck that increased the training time. To mitigate this problem, we proposed a Custom Training Loop that instead of synchronizing the parameters between nodes in every batch, only synchronizes at a interval of batches, then resulting in a lower synchronization overhead. To this work we defined this interval of batches as 4 batches.

3.4. Design of Experiments (DoE)

We selected two different Deep Learning models to train: **ResNet50** [He et al. 2015] as a big model with ~25.6M parameters and **EfficientNetB0** [Tan and Le 2019] as a small model with ~5.3M parameters. The two networks were trained using the TinyImageNet-200, a subset of the ImageNet [Deng et al. 2009] containing 100000 downsized to a 64x64 colored image divided among 200 classes. We created a full factorial for each model with varying batch size, the number of nodes used and the type of the training. The levels for the batch size factor were 32, 64 and 100, for the nodes we trained with 1, 2 and 3 nodes and for the type of training we had "sync" that uses the `model.fit` method to train or "ctl" that uses our custom training loop to train. This is a total of 2x3x3x2 different executions, each execution as replicated 5 times in a random order. Our output variable was the training time (full training time and time per epoch).

4. Results

Figure 1 show a full view of our results. The graph presents the mean training time in minutes as a function of the number of nodes, faceted by type and batch size and with the

color representing the model. Each bar represents the mean for the five executions for each configuration. The black transparent dots represents our observations. This allow us to see two main results: The first one is that the inclusion of more then one node introduces the necessity of communication, and together communication overhead. This overhead can be better seen with the smaller batches, as the computation is faster and more communication is needed. We can also see that the ResNet50 shows a higher time then the EfficientNetB0 in this cases. This happens because all the parameters needed to be synchronized after every batch, and the ResNet50 is almost 5 times bigger than the EfficientNetB0. The second one is that the ctl showed a considerable smaller communication overhead even for small batch sizes. The ctl showed **OOM** errors when running with the batch size 100 for training the EfficientNetB0 model.

After the communication overhead introduce by the inclusion of a second node, adding more nodes seemed to decrease the training time. We suppose that with enough nodes, the time gained by computing can overcome the overhead and be better then running it in one machine.

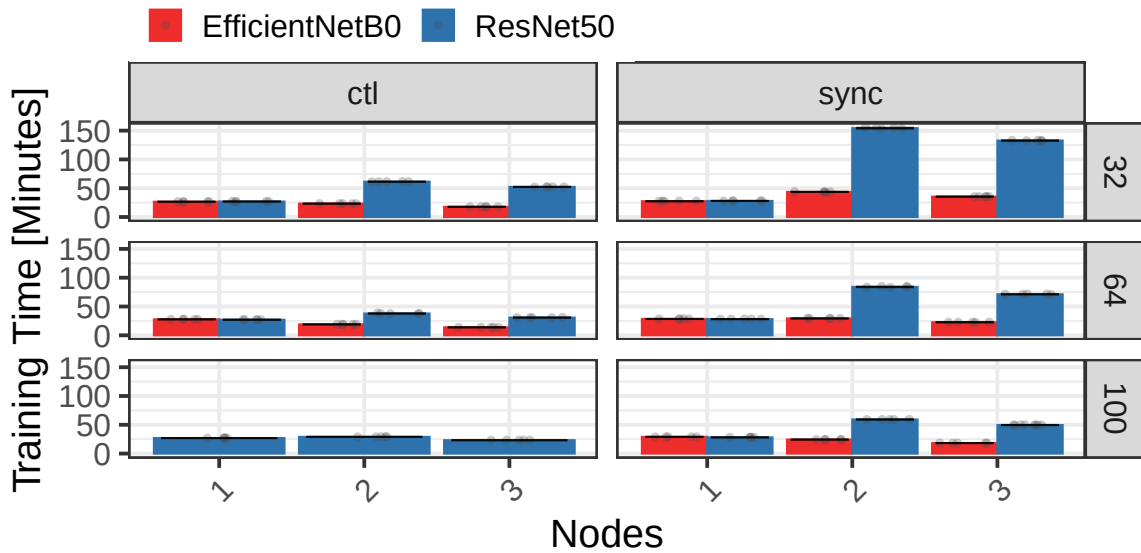


Figure 1. Mean of experimental results.

4.1. The impact of node number and batch size

Figure 2 shows the mean training time grouped by node count. This enable us to see that in fact the increase of more then two machines results in a lower training time even though this gain is small. Figure 3 shows also an interesting result, we can see that increase of the batch size consistently show performance gains, but also decrease the variance. This means that conforms the batch size grows the training time is almost the same despite the node count.

5. Conclusion

We could conclude that the network and the model size have a huge importance in the distributed training, but that even with a network with a low throughput, it is possible to gain performance when training by correctly configuring the environment and model

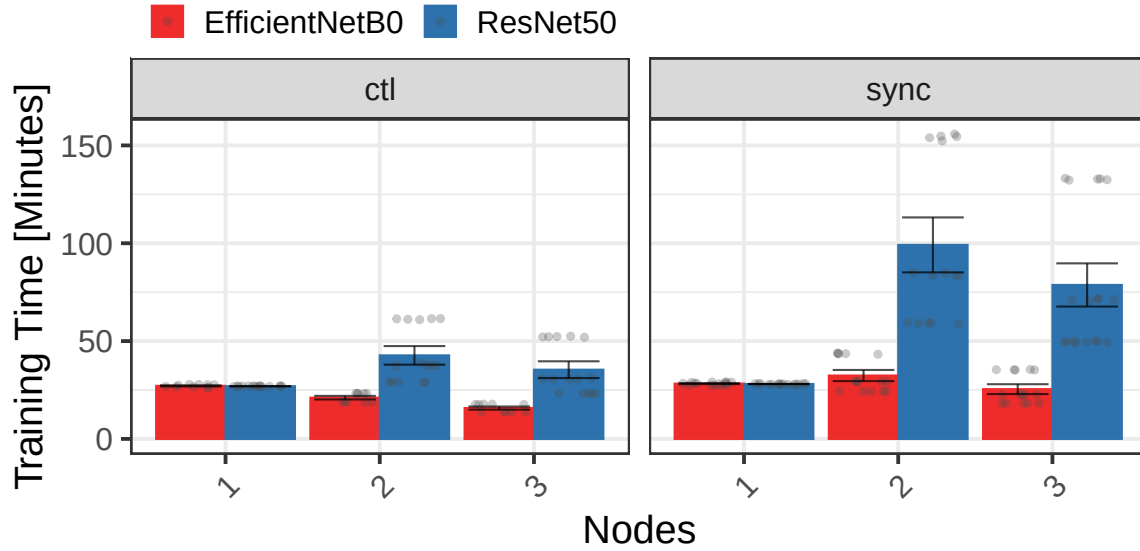


Figure 2. Mean of experimental results grouped by node count.

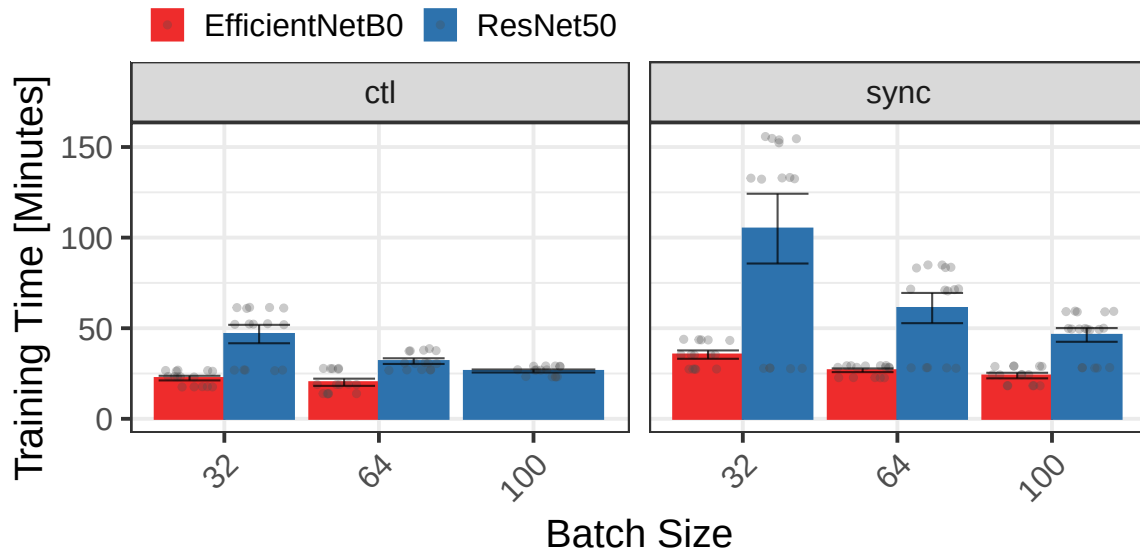


Figure 3. Mean of experimental results grouped by batch size.

parameters. As future work we intend to deep investigate the impact of the network and the model size in the distributed training.

Acknowledgements

We would like to express our gratitude to Professor Dr. Lucas Mello Schnorr for his guidance and the valuable insights shared regarding performance analysis. We also would like to thank the PCAD at INF/UFRGS for making infrastructure and hardware used in the experiments available.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G.,

- Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., and Zheng, X. (2016). Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association.
- Courtès, L. (2013). Functional package management with guix. In *Proceedings of the 6th European Lisp Symposium (ELS)*, pages 15–28.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee.
- Documentation, T. (2023). Distributed training with keras. <https://www.tensorflow.org/tutorials/distribute/keras>. Accessed: 2025-11-30.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Deep residual learning for image recognition. *CoRR*, abs/1512.03385.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946.
- Yoo, A. B., Jette, M. A., and Grondona, M. (2003). Slurm: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing*, pages 44–60. Springer.